DevOps Infrastructure – Introduction to Containers & Evolution

---

1. What are Containers?

A **container** is a lightweight, standalone, executable software package that includes everything needed to run an application:

- Code
- Runtime
- Libraries
- Dependencies
- Configuration files

Key Idea:

Containers allow applications to run **consistently across different environments** (development, testing, production).

Example:

A Python app inside a container will run the same on:

- Your laptop
- A cloud server
- A CI/CD pipeline

---

2. Origin of Containers

Containers did not appear suddenly—they evolved from earlier technologies.

Timeline:

a) Traditional Deployment (Before Virtualization)

- Applications installed directly on OS
- Problems:
    - Dependency conflicts
    - Hard to scale
    - Environment mismatch

b) Virtual Machines (VMs)

- Each VM runs:
  - Full OS
  - Hypervisor-based isolation
- Advantages:
  - Strong isolation
- Disadvantages:
  - Heavy (GBs in size)
  - Slow startup
  - Resource-intensive

c) Early Container Concepts

- Technologies like:
  - **chroot (Unix)** – isolated file system
  - **LXC (Linux Containers)** – OS-level virtualization

---

3. Emergence of Modern Containerization

Modern containers became popular with the introduction of **Docker (2013)**.

Why Containers Became Popular:

- Lightweight compared to VMs
- Fast startup (seconds)
- Efficient resource usage
- Easy portability

Key Innovation:

Instead of virtualizing **hardware**, containers virtualize the **OS layer**.

---

4. Containers vs Virtual Machines

Feature	Containers	Virtual Machines
Size	MBs	GBs

Feature	Containers	Virtual Machines
Startup Time	Seconds	Minutes
OS	Shared host OS	Separate OS per VM
Performance	Near native	Slight overhead
Isolation	Process-level	Full isolation

---

5. Integration of Containers into DevOps

Containers are a core part of **DevOps workflows**.

How Containers Help DevOps:

Consistency

- "Works on my machine" problem solved

CI/CD Pipelines

- Same container used in:
    - Build → Test → Deploy

Microservices Architecture

- Each service runs in its own container

Scalability

- Containers can be scaled horizontally

Automation

- Easily integrated with tools like:
    - Kubernetes
    - Jenkins
    - GitHub Actions

---

6. Real-World Use Case

Example: Web Application Deployment

Without containers:

- Install dependencies manually
- Environment mismatch issues

With containers:

- Build once → run anywhere
- Same image deployed everywhere

---

7. Summary

- Containers package applications with dependencies
- Evolved from chroot → LXC → Docker
- Lightweight alternative to virtual machines
- Core component of modern DevOps practices
- Enable portability, scalability, and automation